Algorithm

Binary Tree Node Counting Using C++ : step-by-step algorithm with trace, proof and complexity

1. Aim of the Algorithm

The aim is to visit every node of a binary tree exactly once and, during the visit, update four counters: the **total number of nodes**, the number of **leaf nodes** (no children), the number of **internal nodes** (at least one child) and the number of nodes having **exactly one child**. The traversal is done with recursion, and each node is classified by looking only at its left and right pointers.

2. Input, Output and Assumptions

Item	Description
Input	Pointer `root` to the root node of a binary tree (the tree may be empty, in which case `root = NULL`).
Output	Four integers: total nodes, leaf nodes, internal nodes and nodes with exactly one child.
Assumption 1	Every node has a data field and two pointers, `left` and `right`; a missing child is stored as `NULL`.
Assumption 2	The tree contains no cycles, so the recursion always terminates.
Assumption 3	The values stored in the nodes do not affect the counts; only the shape of the tree matters.

3. Data Structure Used

A binary tree node is represented with a C++ structure that holds the data and two pointers to the child nodes.

```
struct Node {
    int data;       // value stored in the node
    Node *left;     // pointer to the left child  (NULL if absent)
    Node *right;    // pointer to the right child (NULL if absent)
};
```

4. Variables Used

Variable	Type	Purpose	Initial value
`root`	Node*	Pointer to the current node being processed (starts at the real root)	Root of tree
`total`	int (by reference)	Counts every node in the tree	0
`leaf`	int (by reference)	Counts nodes with 0 children	0
`internal`	int (by reference)	Counts nodes with 1 or 2 children	0
`oneChild`	int (by reference)	Counts nodes with exactly 1 child	0

Why by reference? All recursive calls must update the same four counters. If the counters were passed by value, each call would change only its own copy and `main()` would always print 0.

5. Node Classification Rule

Only two pointer tests are needed. The table shows what happens for every possible combination.

left == NULL ?	right == NULL ?	Type of node	total	leaf	internal	oneChild
Yes	Yes	Leaf node	+1	+1	0	0
Yes	No	One child (right only)	+1	0	+1	+1
No	Yes	One child (left only)	+1	0	+1	+1
No	No	Two children	+1	0	+1	0

6. Algorithm A : Main Program

1. **Start.**
2. Define the node structure with `data`, `left` and `right`.
3. Create the binary tree by allocating nodes (`new Node`) and linking them through their left and right pointers.
4. Declare and initialize `total = 0`, `leaf = 0`, `internal = 0` and `oneChild = 0`.
5. Call `countNodes(root, total, leaf, internal, oneChild)`.
6. Display the values of `total`, `leaf`, `internal` and `oneChild`.
7. Free the memory used by the tree (optional but recommended).
8. **Stop.**

7. Algorithm B : countNodes(node) (Recursive)

Step	Action	Explanation
1	`IF node == NULL THEN RETURN`	Base case. An empty subtree has no nodes, so nothing is counted and the recursion stops here.
2	`total = total + 1`	The node exists, so it is counted. Every non-NULL node reaches this step exactly once.
3	`IF node.left == NULL AND node.right == NULL`	Checks for a leaf: the node has neither a left nor a right child.
4	`leaf = leaf + 1`	The node is a leaf, so the leaf counter increases. Control then goes to step 7.
5	`ELSE internal = internal + 1`	At least one child exists, so the node is an internal node.
6	`IF node.left == NULL OR node.right == NULL THEN oneChild = oneChild + 1`	Inside the internal case, if one of the pointers is NULL then exactly one child exists. The case 'both NULL' was already handled in step 3, so OR is correct here.
7	`CountNodes(node.left)`	Process the entire left subtree.
8	`CountNodes(node.right)`	Process the entire right subtree. When it returns, the whole subtree of this node has been counted.

8. Pseudocode

```
Algorithm CountNodes(node)
    if node is NULL then
        return
    end if
    total <- total + 1
    if node.left is NULL and node.right is NULL then
        leaf <- leaf + 1
    else
        internal <- internal + 1
        if node.left is NULL or node.right is NULL then
            oneChild <- oneChild + 1
        end if
    end if
    CountNodes(node.left)
    CountNodes(node.right)
end Algorithm
```

9. Dry Run on an Example Tree

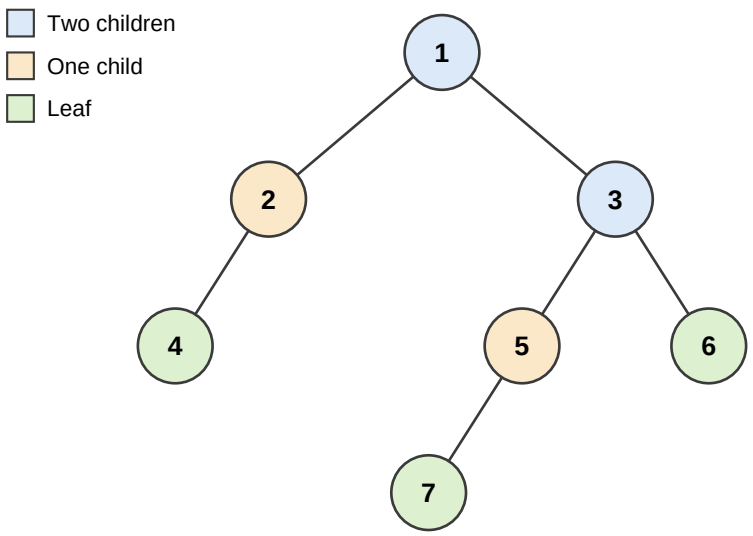

Figure 1: Example binary tree (colour shows the type of each node)

Example tree used in all documents: node 1 is the root with children 2 and 3; node 2 has only a left child (4); node 3 has children 5 and 6; node 5 has only a left child (7).

The algorithm visits the nodes in **preorder** (Root, Left, Right): **1, 2, 4, 3, 5, 7, 6**. The table shows the counters after each node is processed.

Step	Node visited	Left child	Right child	Type	total	leaf	internal	oneChild
1	1	2	3	Two children	1	0	1	0
2	2	4	NULL	One child	2	0	2	1
3	4	NULL	NULL	Leaf	3	1	2	1
4	3	5	6	Two children	4	1	3	1
5	5	7	NULL	One child	5	1	4	2
6	7	NULL	NULL	Leaf	6	2	4	2
7	6	NULL	NULL	Leaf	7	3	4	2

Final result: total = 7, leaf = 3 (nodes 4, 7, 6), internal = 4 (nodes 1, 2, 3, 5), nodes with exactly one child = 2 (nodes 2 and 5). Check: leaf (3) = nodes with two children (2) + 1 = 3, which is correct.

10. Recursive Call Sequence

The function is called once for every node and once for every missing child (a NULL pointer). A tree with n nodes has n + 1 NULL pointers, so there are 2n + 1 calls in total; here 2 x 7 + 1 = 15.

Call	Argument	What happens
1	node 1	total = 1; two children, so internal = 1; calls left (2) then right (3)
2	node 2	total = 2; internal = 2; oneChild = 1 (only left child 4)
3	node 4	total = 3; leaf = 1
4 - 5	NULL, NULL	Left and right of node 4 return immediately (base case)

Call	Argument	What happens
6	NULL	Right child of node 2 returns immediately
7	node 3	total = 4; internal = 3; calls left (5) then right (6)
8	node 5	total = 5; internal = 4; oneChild = 2 (only left child 7)
9	node 7	total = 6; leaf = 2
10 - 11	NULL, NULL	Children of node 7 return immediately
12	NULL	Right child of node 5 returns immediately
13	node 6	total = 7; leaf = 3
14 - 15	NULL, NULL	Children of node 6 return immediately; the recursion unwinds to main()

11. Proof of Correctness

Claim: after `CountNodes(root)` returns, the four counters hold the exact counts for the whole tree.

Proof by induction on the number of nodes.

- **Base case:** for an empty tree (`NULL`) the function returns without changing anything, which is correct because there are 0 nodes.
- **Inductive step:** assume the function counts any tree with fewer than n nodes correctly. For a tree with n nodes, the call on the root adds exactly 1 to `total`, adds exactly 1 to `leaf` or `internal` (never both), and adds 1 to `oneChild` only when exactly one pointer is NULL. The left and right subtrees have fewer than n nodes, so by the assumption the two recursive calls count them correctly. The root, the left subtree and the right subtree are disjoint and together form the whole tree, so the final counters are correct.

Useful identities for self-checking

Identity	Meaning	Example (n = 7)
total = leaf + internal	Every node is either a leaf or an internal node	7 = 3 + 4
internal = oneChild + twoChild	Internal nodes have one or two children	4 = 2 + 2
leaf = twoChild + 1	Holds for every non-empty binary tree	3 = 2 + 1
NULL calls = n + 1	Number of empty pointers in the tree	8 = 7 + 1

12. Time and Space Complexity

Measure	Complexity	Reason
Time (best, average, worst)	O(n)	Each of the n nodes is visited exactly once and does a constant amount of work (a few comparisons and increments).
Auxiliary space (balanced tree)	O(log n)	Recursion depth equals the tree height h, which is about log n.
Auxiliary space (skewed tree)	O(n)	Height h = n, so n function calls are stored on the call stack at the deepest point.
Auxiliary space (general)	O(h)	Only the stack frames along one root-to-leaf path exist at any moment; the counters are three or four integers.

13. Alternative Algorithm : Iterative (Level Order using Queue)

Recursion can be replaced by a queue. This version avoids deep recursion and cannot overflow the call stack on very tall trees.

1. Initialize all four counters to 0. If `root == NULL`, stop and print zeros.
2. Create an empty queue and push `root`.
3. While the queue is not empty: pop the front node `cur` and do `total++`.
4. If `cur.left == NULL AND cur.right == NULL` then `leaf++`. Otherwise `internal++` and, if exactly one child exists, `oneChild++`.
5. Push the left child if it exists, then push the right child if it exists.
6. When the queue becomes empty, print the four counters.

```
void countIterative(Node* root, int &total, int &leaf,
                    int &internal, int &oneChild) {
    if (root == NULL) return;
    queue<Node*> q;
    q.push(root);
    while (!q.empty()) {
        Node* cur = q.front();  q.pop();
        total++;
        if (!cur->left && !cur->right) leaf++;
        else {
            internal++;
            if (!cur->left || !cur->right) oneChild++;
        }
        if (cur->left)  q.push(cur->left);
        if (cur->right) q.push(cur->right);
    }
}
```

Queue behaviour on the example tree (nodes are processed level by level: 1, 2, 3, 4, 5, 6, 7):

Node popped	Queue after pushing children	total	leaf	internal	oneChild
1	[2, 3]	1	0	1	0
2	[3, 4]	2	0	2	1
3	[4, 5, 6]	3	0	3	1
4	[5, 6]	4	1	3	1
5	[6, 7]	5	1	4	2
6	[7]	6	2	4	2

Node popped	Queue after pushing children	total	leaf	internal	oneChild
7	[ ]	7	3	4	2

The final counters are the same as in the recursive version, so both algorithms are correct.

14. Recursive versus Iterative

Point	Recursive (preorder)	Iterative (queue)
Code length	Shorter and easier to read	Slightly longer
Extra memory	O(h) call stack	O(w) queue, where w is the maximum width of the tree
Stack overflow risk	Possible for very deep (skewed) trees	None
Visiting order	Root, Left, Right	Level by level
Time complexity	O(n)	O(n)
Best used when	The tree is reasonably balanced; clarity is the goal	The tree can be very tall or recursion depth is limited

15. Edge Cases

Tree	total	leaf	internal	oneChild	Comment
Empty tree (root = NULL)	0	0	0	0	Base case returns immediately
Single node	1	1	0	0	The root is a leaf
Left-skewed, 4 nodes	4	1	3	3	Every internal node has one child
Right-skewed, 4 nodes	4	1	3	3	Same counts, mirror image
Perfect tree, 7 nodes	7	4	3	0	No node has exactly one child

16. Common Mistakes to Avoid

- Forgetting the `NULL` base case, which causes infinite recursion and a crash.
- Passing the counters by value instead of by reference (the result stays 0).
- Counting a one-child node as a leaf, or forgetting that it is also an internal node.
- Using `&&` instead of `||` in the one-child test; `&&` would test for two NULL children, which is the leaf case.
- Not initializing the counters to 0 before the first call.
- Confusing 'internal nodes' with 'nodes having two children'. Internal nodes = one-child nodes + two-child nodes.

17. Summary

A single preorder traversal is enough to count all four categories. At each node the algorithm increments the total, checks the two child pointers to classify the node, and then recurses into the left and right subtrees. It runs in O(n) time and uses O(h) auxiliary space, and its result can always be verified with the identity *leaf = nodes with two children + 1*.